

SQL Optimization Documentation - Logistics ERP

1. Introduction

This document provides a comprehensive overview of SQL optimization techniques applied to the logistics_erp dataset.

It includes indexing strategies, query optimization, execution plans, partitioning, and performance tuning methods.

2. Query Optimization

Query optimization focuses on improving SQL query performance.

Avoid SELECT * and fetch only required columns.

Use WHERE clauses to filter data early.

Minimize joins and use proper join conditions.

Use EXISTS instead of IN when dealing with large datasets.

3. Indexing Strategy

Indexes improve data retrieval speed.

Clustered Index: Defines physical storage order (Primary Key).

Non-Clustered Index: Separate structure pointing to data rows.

Example:

```
CREATE INDEX idx_customer ON logistics_erp(customer_id);
```

```
CREATE INDEX idx_product ON logistics_erp(product_id);
```

4. Covering Index

A covering index includes all columns required by a query.

This avoids accessing the base table.

Example:

```
CREATE INDEX idx_cover ON logistics_erp(customer_id, order_id, order_value);
```

5. Query Execution Plan

Use EXPLAIN to analyze query performance.

It shows join types, indexes used, and rows scanned.

Example:

```
EXPLAIN SELECT * FROM logistics_erp WHERE customer_id='CUST001';
```

6. Partitioning Strategy

Partitioning splits large tables into smaller pieces.

Improves query performance on large datasets.

Example:

```
PARTITION BY RANGE (YEAR(order_date));
```

7. Statistics Optimization

Database optimizer relies on table statistics.

Update statistics regularly.

Example:

```
ANALYZE TABLE logistics_erp;
```

8. Locking Analysis

Locks control concurrent data access.

Row-level locking is preferred.

Use shorter transactions to reduce locking issues.

9. Deadlock Prevention

Deadlocks occur when two transactions wait on each other.

Always access tables in same order.

Keep transactions short and efficient.

Use proper indexing to avoid locking large data.

10. Query Benchmarking

Measure query performance before and after optimization.

Use profiling tools.

Example:

SET profiling = 1;

SHOW PROFILES;